

Outline

- First C Program
- Compilation Process in C
- The main() function
- #define directive
- #include directive
- Character set
- Keywords
- Identifiers
- Tokens
- Constants
- Data Types
- Variables
- Declaring and Initializing Variables
- Input/Output in C
- Formatting Output

First C Program

```
#include <stdio.h>

int main() {
    /* my first program in C */
    printf("Hello, World! \n");

    return 0;
}
```

Let us take a look at the various parts of the above program –

- The first line of the program `#include <stdio.h>` is a preprocessor command, which tells a C compiler to include `stdio.h` file before going to actual compilation.
- The next line `int main()` is the main function where the program execution begins.
- The next line `/*...*/` will be ignored by the compiler and it has been put to add additional comments in the program. So such lines are called comments in the program.
- The next line `printf(...)` is another function available in C which causes the message "Hello, World!" to be displayed on the screen.
- The next line `return 0;` terminates the `main()` function and returns the value 0.

Compilation Process in C

- The compilation is a process of converting the source code into object code.
- It is done with the help of the compiler.
- The compiler checks the source code for the syntactical or structural errors, and if the source code is error-free, then it generates the object code.

```
#include<stdio.h>
main()
{
printf("Hello javaTpoint");
return 0;
}
```

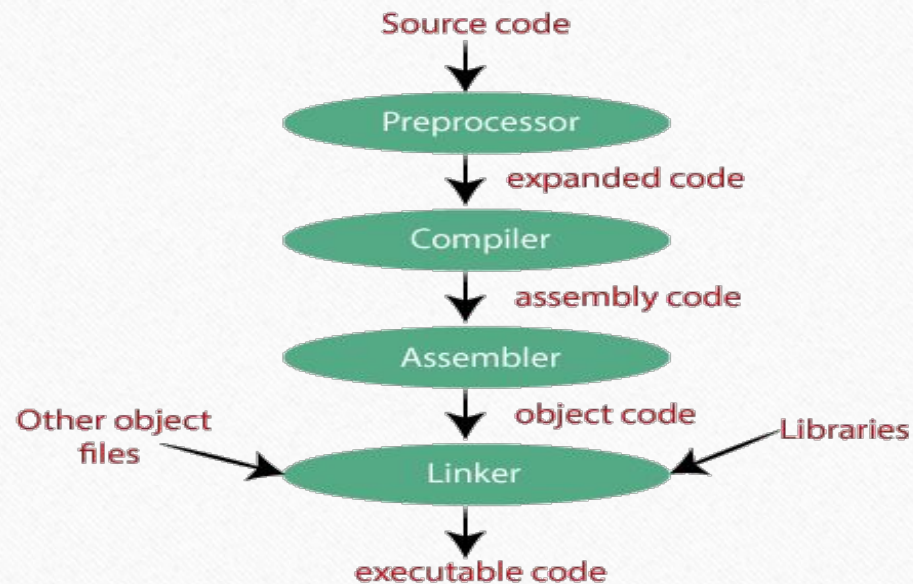


```
0100000000000000
0111111111111111
01010101101010
00000011111111
00000111111111
00000010101011
```

Compilation Process in C cont..

The following are the phases through which our program passes before being transformed into an executable form:

- **Preprocessor**
- **Compiler**
- **Assembler**
- **Linker**



The main Function

The main is a part of a every C program. C permits different forms of main statement. Following forms are allowed.

`main()`

`int main()`

`void main()`

`main(void)`

`void main(void)`

`int main(void)`

The main Function

- The empty pair of parentheses indicates that the function has no arguments.
- This may be explicitly indicated by using the keyword `void` inside the parentheses.
- We may also specify the keyword `int` or `void` before the word `main`.
- The keyword `void` means that the function does not return any information to the operating system and `int` means that the function returns an integer value to the operating system.
- When `int` is specified, the last statement in the program must be `“return 0”`

The #define Directive

- A #define is a preprocessor compiler directive.
- Therefore #define lines should not end with a semicolon.
- If we want to define values to a symbolic constant and to define any statements which uses more frequently in the program we use the #define directive.
- A constant is defined as follows:
 - **Syntax :** #define symbolic-name value of constant
 - **Example :** #define NUMBER 10

The #include Directive

- C programs are divided into modules or functions.
- Some functions are written by users, like us, and many others are stored in the C library.
- Library functions are grouped category-wise and stored in different files known as header files.
- If we want to access the functions stored in the library, it is necessary to tell the compiler about the files to be accessed.
- This is achieved by using the preprocessor directive `#include` as follows:
 - **Syntax :** `#include<filename>`
 - **Example :** `#include<stdio.h>`

Character Set

- A character set is a set of alphabets, letters and some special characters that are valid in C language.
- C language supports total 256 characters.

- **Alphabets**

- Uppercase: A B C X Y Z

- Lowercase: a b c x y z

C accepts both lowercase and uppercase alphabets as variables and functions.

- **Digits**

0 1 2 3 4 5 6 7 8 9

Character Set cont..

Special Characters

Special Characters in C Programming				
,	<	>	.	-
(	)	;	\$	:
%	[	]	#	?
'	&	{	}	"
^	!	*	/	
-	\	_	+	

White space Characters

Blank space, newline, horizontal tab, carriage return and form feed.

C Keywords

- Keywords are predefined, reserved words used in programming that have special meanings to the compiler.
- Keywords are part of the syntax and they cannot be used as an identifier.
- There are total 32 keywords in C programming language.
- Keywords are sometimes also called reserved words.
- **For example:**
 - `int money;`
 - Here, `int` is a keyword that indicates `money` is a variable of type `int` (integer).

As C is a case sensitive language, all keywords must be written in lowercase.

C Keywords List cont..

C Keywords			
auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
continue	for	signed	void
do	if	static	while
default	goto	sizeof	volatile
const	float	short	unsigned

C Identifiers

- Identifier refers to name given to entities such as variables, functions, structures etc.
- Identifiers must be unique.
- They are created to give a unique name to an entity to identify it during the execution of the program.
- **For example:**
 - `int money;`
 - `double accountBalance;`
- Here, `money` and `accountBalance` are identifiers.
- Also remember, identifier names must be different from keywords.
- You cannot use `int` as an identifier because `int` is a keyword.

C Identifiers cont..

Rules for naming identifiers

- A valid identifier can have letters (both uppercase and lowercase letters), digits and underscores.
- The first letter of an identifier should be either a letter or an underscore.
- You cannot use keywords like int, while etc. as identifiers.
- No space is allowed in identifiers.
- There is no rule on how long an identifier can be. However, you may run into problems in some compilers if the identifier is longer than 31 characters.
- You can choose any name as an identifier if you follow the above rule, however, give meaningful names to identifiers that make sense.

C Identifiers cont..

Check which of the following identifiers are valid:

- Sum01 – Valid
- _name – Valid
- 01_name – Invalid
- Simple interest – Invalid
- Test_ide_001 – Valid
- int - Invalid
- Int - Valid

Tokens

- Tokens in C is the most important element to be used in creating a program in C.
- We can define the token as the smallest individual element in C.
- For example, we cannot create a sentence without using words; similarly, we cannot create a program in C without using tokens in C.
- Therefore, we can say that tokens in C is the building block or the basic component for creating a program in C language.

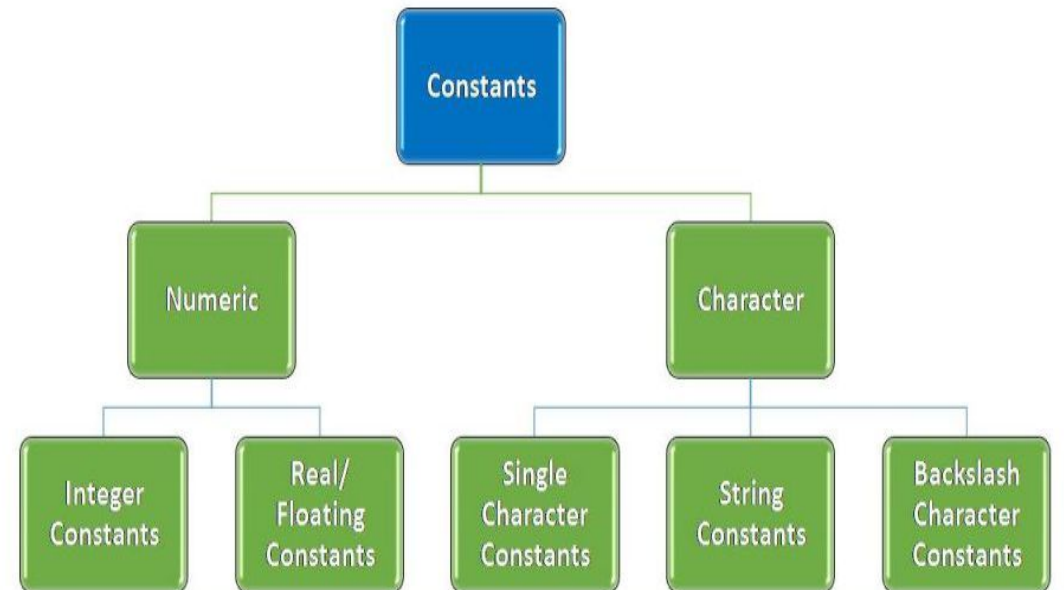
Tokens cont..

Classifications of Tokens :

- **Keywords in C**
 - int, float, char etc..
- **Identifiers in C**
 - Rahim, karim, ram etc...
- **Strings in C**
 - “ABC”, “A” etc...
- **White space**
 - Blank space, tab, \n etc..
- **Operators in C**
 - +, -, / etc...
- **Constant in C**
 - 5, 7.5, ‘A’ etc..
- **Special Characters in C**
 - (,), {, }, ; etc...

C Constants and Literals

- A constant is fixed a value, which cannot change the meaning of value during program execution. These fixed values are also called literals.



Numeric Constants – Integer Constants

- A sequence of digits without decimal point or exponential part (or both). i.e -5, 10, -1743 etc.
- Rules for writing integer constants :
 - Must have atleast one digit without decimal point. 5, 7, -9 etc
 - Commas and blank space are not allowed. 7,000, 1 00 00 etc are not allowed.
 - Integer can be positive (+) or negative (-). Default sign is +

Numeric Constants – Integer Constants

- There are three representation of integer constants :
 - Decimal Integer Constant (Base 10)
 - Digit range 0 to 9
 - **Example** : 347, 19, 158 etc
 - Octal Integer Constant (Base 8)
 - Digit range 0 to 7
 - Identifying by a leading 0
 - **Example** : 0347, 0546 etc.
 - Hexadecimal Integer Constant (Base 16)
 - Digit range 0 to 9 and A to F for 10 to 15
 - Identifying by leading 0x or 0X
 - **Example** : 0x125A, 0x175 etc.

Numeric Constants – Floating Point Constants

- A floating-point literal is a numeric literal that has either a fractional form or an exponent form.
- **For example:**
 - -2.0
 - 0.0000234
 - -0.22E-5
 - Note: $E-5 = 10^{-5}$

Character – Single Character Constants

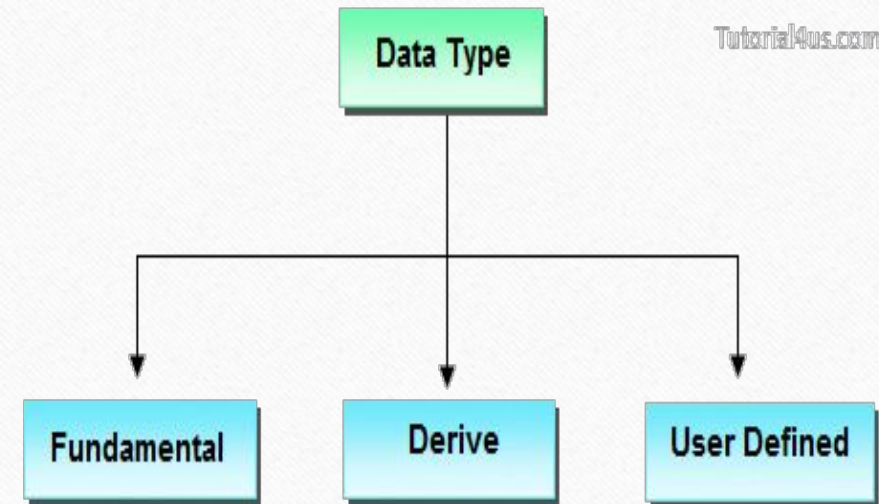
- It contains single character enclosed in single quotation mark (‘ ’).
- The character may be a alphabet, digit or special characters.
- **Example** : ‘a’, ‘A’, ‘9’, ‘@’, ‘\$’ etc.

Character – String Constants

- It contains any number of characters enclosed in double quote (“ ”).
- **Example :** “A”, “a” etc..

Data Types

- **Data type** is a keyword used to identify type of data.
- In general every programming language is containing three categories of data types. They are :
 - Fundamental or primitive data types
 - Derived data types
 - User defined data types



Data Types cont..

Fundamental or Primitive Data Types :

- These are the data types whose variable can hold maximum one value at a time, in C language it can be achieved by int, float, double, char.

Data Types cont..

Integers

- Integers are whole numbers that can have both zero, positive and negative values but no decimal values.
- For example, 0, -5, 10
- We can use `int` for declaring an integer variable.

float and double

- float and double are used to hold real numbers.
- In C, floating-point numbers can also be represented in exponential.
- For example,
 - `float normalizationFactor = 22.442e2;`

Data Types cont..

Char

- Keyword char is used for declaring character type variables.
- For example,
 - `char test = 'h';`
- The size of the character variable is 1 byte.

void

- void is an incomplete type.
- It means "nothing" or "no type".
- You can think of void as absent.
- For example, if a function is not returning anything, its return type should be void.

Note that, you cannot create variables of void type.

Data Types cont..

Derived data types

- These data type are derived from fundamental data type.
- Variables of derived data type allow us to store multiple values of same type in one variable but never allows to store multiple values of different types.
- These are the data type whose variable can hold more than one value of similar type.
- In C language it can be achieve by array, functions, pointer

Example

```
int a[] = {10,20,30}; // valid
int b[] = {100, 'A', "ABC"}; // invalid
```


Data Types cont..

User defined data types

- User defined data types related variables allows us to store multiple values either of same type or different type or both.
- This is a data type whose variable can hold more than one value of dissimilar type, in C language it is achieved by structure, union, enum etc..

Syntax

```
struct emp
{
    int id;
    char ename[10];
    float sal;
};
```

Data Type Modifiers

- In c language Data Type Modifiers are keywords used to change the properties of current properties of data type.
- Data type modifiers are classified into following types.
 - Long
 - Short
 - Unsigned
 - signed
- Modifiers are prefixed with basic data types to modify (either increase or decrease) the amount of storage space allocated to a variable.
- For example, storage space for int data type is 4 byte for 32 bit processor.
- We can increase the range by using long int which is 8 byte.
- We can decrease the range by using short int which is 2 byte.

Data Type Modifiers cont..

long:

- This can be used to increased size of the current data type to 2 more bytes, which can be applied on int or double data types.
- For example int occupy 2 byte of memory if we use long with integer variable then it occupy 4 byte of memory.

long int a; → occupies 4 bytes of memory space

↓ ↓

2 + 2 → 4 bytes

long double b; → occupies 10 bytes of memory space

↓ ↓

2 + 8 → 10 byte

Data Type Modifiers cont..

short

In general int data type occupies different memory spaces for a different operating system; to allocate fixed memory space short keyword can be used.

Syntax

```
short int a; --> occupies 2 bytes of memory space in every operating system.
```


Data Type Modifiers cont..

unsigned

This keyword can be used to make the accepting values of a data type is positive data type.

Syntax

```
unsigned int a =100;    // right
unsigned int a=-100;    // wrong
```

Data Type Modifiers cont..

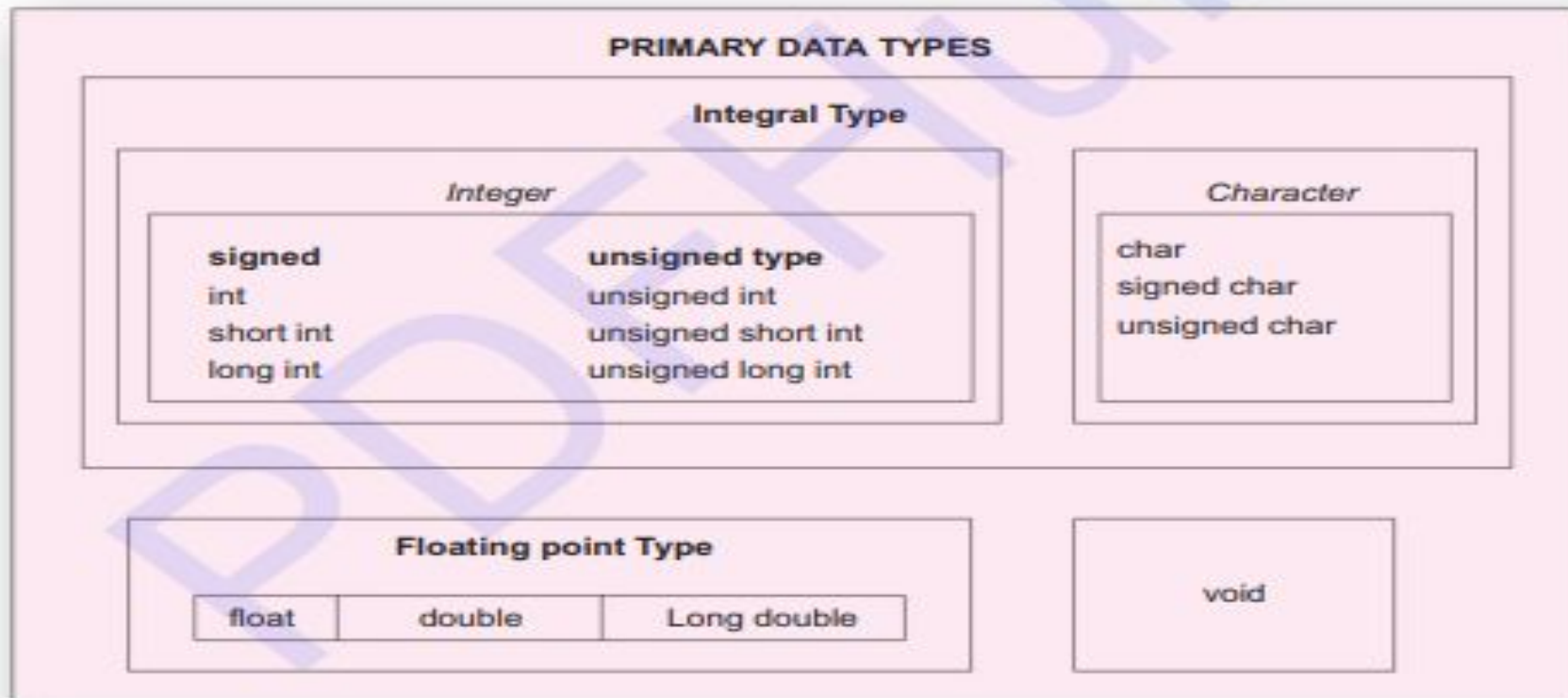
Signed

This keyword accepts both negative or positive value and this is default properties or data type modifiers for every data type.

Example

```
int a=10;           // right
int a=-10;          // right
signed int a=10;    // right
signed int a=-10;   // right
```


Data Types cont..



Data Types cont..

Table 2.8 Size and Range of Data Types on a 16-bit Machine

Type	Size (bits)	Range
char or signed char	8	-128 to 127
unsigned char	8	0 to 255
int or signed int	16	-32,768 to 32,767
unsigned int	16	0 to 65535
short int or		
signed short int	8	-128 to 127
unsigned short int	8	0 to 255
long int or		
signed long int	32	-2,147,483,648 to 2,147,483,647
unsigned long int	32	0 to 4,294,967,295
float	32	$3.4E - 38$ to $3.4E + 38$
double	64	$1.7E - 308$ to $1.7E + 308$
long double	80	$3.4E - 4932$ to $1.1E + 4932$

Program to Show Size of Data Types

C Program to Find Size of Data Types

```
#include<stdio.h>
int main()
{
    printf("Size of short is %ld bytes\n",sizeof(short));
    printf("Size of int is %ld bytes\n",sizeof(int));
    printf("Size of long is %ld bytes\n",sizeof(long));

    printf("Size of float is %ld bytes\n",sizeof(float));
    printf("Size of double is %ld bytes\n",sizeof(double));
    printf("Size of long double is %ld bytes\n",sizeof(long double));

    printf("Size of char is %ld bytes\n",sizeof(char));
    printf("Size of void is %ld bytes\n",sizeof(void));
    return 0;
}
```

C Variables

- In programming, a variable is a container (storage area) to hold data.
- To indicate the storage area, each variable should be given a unique name (identifier).
- Variable names are just the symbolic representation of a memory location.
 - For example:
 - `int playerScore = 95;`
 - Here, `playerScore` is a variable of `int` type.
 - Here, the variable is assigned an integer value 95.

C Variables cont..

Rules for naming a variable

- A variable name can only have letters (both uppercase and lowercase letters), digits and underscore.
- The first letter of a variable should be either a letter or an underscore.
- There is no rule on how long a variable name (identifier) can be. However, you may run into problems in some compilers if the variable name is longer than 31 characters.
- **Note:** You should always try to give meaningful names to variables. For example: firstName is a better variable name than fn.

Declaration of Variable

- Variables are declared at the beginning of the body but after the main.
- All variables must be declared before they can appear in executable statements.
- The syntax for declaring a variable is as follow: Data-type v1,v2,.....vn;
- v1, v2,.....vn are the names of variables.
- For example, valid declarations are
 - int count;
 - int number,total;
 - float ratio;

Difference between Variables and Symbolic Names

- A variable may be used to store data value.
- A variable may take different values at different times during execution of a program. Variables need to declare at the beginning of the body but after the main.
- Symbolic names are unique constants.
- These constants may appear in a number of place in the program.
- Symbolic names need to be defined at the beginning of a program.

Initialization of Variable

- The process of giving initial values to variables is called initialization.
- Example of initialization :
 - `int a=100;`
 - `char name = 'x';`
 - `float number =75.84 ;`

C input and output

- The `printf()` and `scanf()` functions are used for input and output in C language.
- Both functions are inbuilt library functions, defined in `stdio.h` (header file).

C output

In C programming, `printf()` is one of the main output function. The function sends formatted output to the screen. For example,

Example 1: C Output

```
#include <stdio.h>
int main()
{
    // Displays the string inside quotations
    printf("C Programming");
    return 0;
}
```



Run Code »

Output

```
C Programming
```


C output cont..

Example 2: Integer Output

```
#include <stdio.h>
int main()
{
    int testInteger = 5;
    printf("Number = %d", testInteger);
    return 0;
}
```



Run Code >>

Output

Number = 5

C output cont..

Example 3: float and double Output

```
#include <stdio.h>
int main()
{
    float number1 = 13.5;
    double number2 = 12.4;

    printf("number1 = %f\n", number1);
    printf("number2 = %lf", number2);
    return 0;
}
```

Run Code >>

Output

```
number1 = 13.500000
number2 = 12.400000
```


C output cont..

Example 4: Print Characters

```
#include <stdio.h>
int main()
{
    char chr = 'a';
    printf("character = %c", chr);
    return 0;
}
```



Run Code >>

Output

```
character = a
```

C Input

- In C programming, `scanf()` is one of the commonly used function to take input from the user.
- The `scanf()` function reads formatted input from the standard input such as keyboards.

C Input cont..

Example 5: Integer Input/Output

```
#include <stdio.h>
int main()
{
    int testInteger;
    printf("Enter an integer: ");
    scanf("%d", &testInteger);
    printf("Number = %d", testInteger);
    return 0;
}
```

Run Code »

Output

```
Enter an integer: 4
Number = 4
```

- Here, we have used %d format specifier inside the scanf() function to take int input from the user.
- When the user enters an integer, it is stored in the testInteger variable.
- Notice, that we have used &testInteger inside scanf().
- It is because &testInteger gets the address of testInteger, and the value entered by the user is stored in that address.

C Input cont..

Example 6: Float and Double Input/Output

```
#include <stdio.h>
int main()
{
    float num1;
    double num2;

    printf("Enter a number: ");
    scanf("%f", &num1);
    printf("Enter another number: ");
    scanf("%lf", &num2);

    printf("num1 = %f\n", num1);
    printf("num2 = %lf", num2);

    return 0;
}
```

Run Code >>

Output

```
Enter a number: 12.523
Enter another number: 10.2
num1 = 12.523000
num2 = 10.200000
```


C Input cont..

Example 7: C Character I/O

```
#include <stdio.h>
int main()
{
    char chr;
    printf("Enter a character: ");
    scanf("%c",&chr);
    printf("You entered %c.", chr);
    return 0;
}
```

Run Code >>

Output

```
Enter a character: g
You entered g
```

- When a character is entered by the user in the above program, the character itself is not stored. Instead, an integer value (ASCII value) is stored.
- And when we display that value using %c text format, the entered character is displayed. If we use %d to display the character, it's ASCII value is printed.

C Input cont..

I/O Multiple Values

Here's how you can take multiple inputs from the user and display them.

```
#include <stdio.h>
int main()
{
    int a;
    float b;

    printf("Enter integer and then a float: ");

    // Taking multiple inputs
    scanf("%d%f", &a, &b);

    printf("You entered %d and %f", a, b);
    return 0;
}
```

Run Code »

Output

```
Enter integer and then a float: -3
3.4
You entered -3 and 3.400000
```


Print an Integer

C

```
#include <stdio.h>
int main() {
    int num = 10; // Declare and initialize
    printf("The value of num is: %d\n", num);
    return 0;
}
```

Declare and Initialize a Character

c

```
#include <stdio.h>
int main() {
    char ch = 'A'; // Declare and initialize a character
    printf("The character is: %c\n", ch);
    return 0;
}
```


Declare and Initialize a Float

c

```
#include <stdio.h>

int main() {
    float pi = 3.14; // Declare and initialize a float
    printf("Value of pi: %.2f\n", pi);
    return 0;
}
```

Declare and Initialize Variables

c

```
#include <stdio.h>
int main() {
    int a = 10;
    float b = 5.5;
    char c = 'A';
    printf("Integer: %d, Float: %.2f, Character: %c\n", a, b, c);
    return 0;
}
```


Declare and Print Multiple Variables

c

```
#include <stdio.h>

int main() {
    int a = 5, b = 10; // Declare and initialize multiple variables
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

Assign Initial Values and Modify Later

c

```
#include <stdio.h>
int main() {
    int a = 5;
    a = 10; // Change value of variable
    printf("Updated value of a: %d\n", a);
    return 0;
}
```


Assign a Constant Value

c

```
#include <stdio.h>
int main() {
    const float pi = 3.14159; // Declare a constant variable
    printf("Value of pi: %.5f\n", pi);
    return 0;
}
```

Input and Output of Variables

c

```
#include <stdio.h>
int main() {
    int x;
    printf("Enter an integer: ");
    scanf("%d", &x);
    printf("You entered: %d\n", x);
    return 0;
}
```


Find the Size of Data Types

c

```
#include <stdio.h>

int main() {
    printf("Size of int: %lu bytes\n", sizeof(int));
    printf("Size of float: %lu bytes\n", sizeof(float));
    printf("Size of double: %lu bytes\n", sizeof(double));
    printf("Size of char: %lu bytes\n", sizeof(char));
    return 0;
}
```

Thank You

